



BASES TECNOLOGICAS PARA EL SERVICIO PÚBLICO.

PROGRAMACIÓN CON JAVA.

MODULO 4.

4B. Estudio de caso. Archivo definitivo Parte 2. Análisis y
diseño de un programa de tienda departamental

ALUMNO: HURTADO LÓPEZ PEDRO.

PROFESOR: MEDINA PINZÓN ALEXIS GIOVANNI.

Jueves 18 de septiembre del 2025.

Introducción.

Se tiene la necesidad de mejorar en proceso de gestión de productos y clientes de la tienda departamental “Dreams”, debido a su crecimiento en los últimos años.

Se desarrollará un sistema automatizado para registrar las ventas e inventario, ya que actualmente se realiza de forma manual.

El sistema permitirá:

- Registrar productos con su nombre, precio, categoría y stock.
- Administrar los clientes con sus datos principales.
 - Nombre, Correo.
- Registro de ventas.
- Validación de stock actual.
- Cálculo de ventas totales.

Para poder iniciar debemos las **CLASES, OBJETOS, ATRIBUTOS, MÉTODOS** y sus **RELACIONES**. Con lo anterior identificado desarrollaremos el diagrama de clases UML.

Desarrollo.

Identificación de Clases y Objetos.

- **Producto:** Son los artículos que se tienen para la venta.
- **Cliente:** Son los datos que se tienen almacenados de los compradores.
- **Ventas:** Son los productos vendidos o que salen del almacén para cierto cliente.

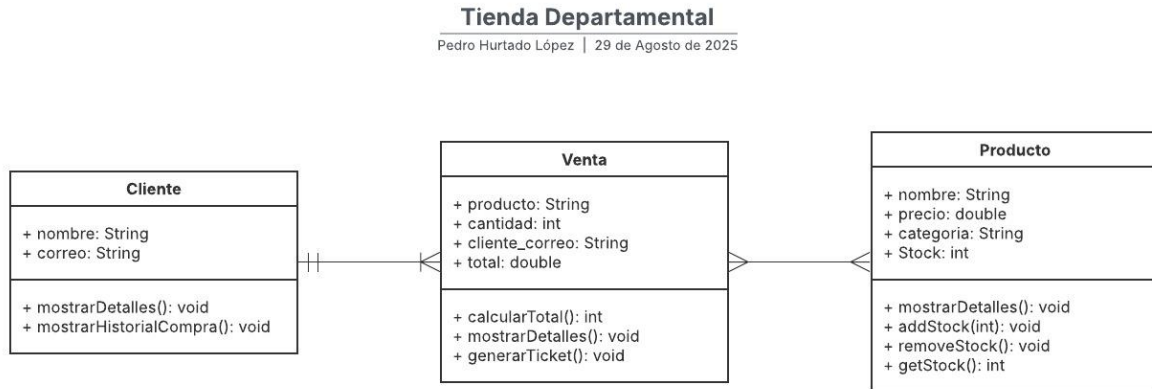
Definición de Atributos y Métodos.

CLASE	ATRIBUTO	MÉTODO
Producto	nombre, precio, categoría, stock.	mostrarDetalles(), addStock(int), removeStock(), getStock()
Cliente	nombre, correo	mostrarDetalles(), mostrarHistorialCompra()
Venta	producto, cantidad, cliente_correo, total.	calcularTotal(), mostrarDetalles(), generarTicket()

Relación entre clases.

- Cada Venta está asociada a un Cliente y a uno o varios Productos.
- Cada Cliente puede estar asociado con una o varias Ventas.
- Cada Producto puede estar asociado a una o varias Ventas.

Diagrama UML.



Conclusiones.

- El ingreso manual de las ventas e inventario es ineficiente, por lo que se tiene que generar un sistema de automatización para optimizar tiempos y tener una mejor precisión o minimizar los errores.
- El sistema propuesto da claridad, estructura y un orden a las ventas, productos y clientes, generando una facilidad en las consultas al tener cada clase atributos y métodos.
- Las ventas van a heredar los datos de los clientes y detalles del productos con el método de `mostrarDetalles()`.
- Se debe generar encapsulamientos con los datos de los clientes para proteger la información de los mismo.
- Para la parte del `calculoTotal()` de las ventas se generaría otro encapsulamiento para tener una mejor precisión en los cálculos, el calculo dependerá de precio y cantidad de productos vendidos.

CODIGO JAVA en VSCode.

TiendaDreams.java

```
import java.util.*;

public class TiendaDreams {
    private static List<Producto> inventario = new ArrayList<>();
    private static List<Cliente> clientes = new ArrayList<>();
    private static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        int opcion;
        do {
            System.out.println("\n--- MENU TIENDA DREAMS ---");
            System.out.println("1. Registrar producto");
            System.out.println("2. Registrar cliente");
            System.out.println("3. Mostrar inventario");
            System.out.println("4. Registrar venta");
            System.out.println("0. Salir");
            System.out.print("Seleccione una opción: ");
            opcion = sc.nextInt();
            sc.nextLine();

            switch (opcion) {
                case 1 -> registrarProducto();
                case 2 -> registrarCliente();
                case 3 -> mostrarInventario();
                case 4 -> registrarVenta();
                case 0 -> System.out.println("Saliendo del sistema...");
                default -> System.out.println("Opción inválida");
            }
        } while (opcion != 0);
    }

    private static void registrarProducto() {
        System.out.print("Nombre del producto: ");
        String nombre = sc.nextLine();
        System.out.print("Precio: ");
        double precio = sc.nextDouble();
        sc.nextLine();
        System.out.print("Categoría: ");
        String cat = sc.nextLine();
        System.out.print("Stock: ");
        int stock = sc.nextInt();
        sc.nextLine();
    }
}
```

```

        inventario.add(new Producto(nombre, precio, cat, stock));
        System.out.println("☑ Producto agregado.");
    }

    private static void registrarCliente() {
        System.out.print("Nombre del cliente: ");
        String nombre = sc.nextLine();
        System.out.print("Correo: ");
        String correo = sc.nextLine();

        clientes.add(new Cliente(nombre, correo));
        System.out.println("☑ Cliente registrado.");
    }

    private static void mostrarInventario() {
        System.out.println("\n--- Inventario ---");
        for (int i = 0; i < inventario.size(); i++) {
            System.out.println((i + 1) + ". " + inventario.get(i));
        }
    }

    private static void registrarVenta() {
        if (clientes.isEmpty() || inventario.isEmpty()) {
            System.out.println("⚠ Debe haber clientes y productos registrados.");
            return;
        }

        System.out.println("\nSeleccione cliente:");
        for (int i = 0; i < clientes.size(); i++) {
            System.out.println((i + 1) + ". " + clientes.get(i));
        }
        int cIdx = sc.nextInt() - 1;
        sc.nextLine();
        Cliente cliente = clientes.get(cIdx);

        Venta venta = new Venta(cliente);

        char continuar;
        do {
            mostrarInventario();
            System.out.print("Seleccione producto: ");
            int pIdx = sc.nextInt() - 1;
            System.out.print("Cantidad: ");

```

```

        int cantidad = sc.nextInt();
        sc.nextLine();

        venta.agregarProducto(inventario.get(pIdx), cantidad);

        System.out.print("¿Agregar otro producto? (s/n): ");
        continuar = sc.nextLine().toLowerCase().charAt(0);
    } while (continuar == 's');

    venta.calcularTotal();
    venta.mostrarFactura();
}
}

```

Cliente.java

```

public class Cliente {
    private String nombre;
    private String correo;

    public Cliente(String nombre, String correo) {
        this.nombre = nombre;
        this.correo = correo;
    }

    public String getNombre() { return nombre; }
    public String getCorreo() { return correo; }

    @Override
    public String toString() {
        return nombre + " (" + correo + ")";
    }
}

```

Producto.java

```

import java.text.DecimalFormat;

public class Producto {
    private String nombre;
    private double precio;
    private String categoria;
}

```

```

private int stock;

public Producto(String nombre, double precio, String categoria, int
stock) {
    this.nombre = nombre;
    this.precio = precio;
    this.categoria = categoria;
    this.stock = stock;
}

public String getNombre() { return nombre; }
public double getPrecio() { return precio; }
public String getCategoria() { return categoria; }
public int getStock() { return stock; }
public void setStock(int stock) { this.stock = stock; }

public void disminuirStock(int cantidad) {
    if (stock >= cantidad) stock -= cantidad;
}

@Override
public String toString() {
    DecimalFormat df = new DecimalFormat("#0.00");
    return nombre + " | $" + df.format(precio) + " | Stock: " + stock;
}
}

```

Venta.java

```

import java.util.ArrayList;
import java.util.List;

public class Venta {
    private Cliente cliente;
    private List<Producto> productos;
    private double total;

    public Venta(Cliente cliente) {
        this.cliente = cliente;
        this.productos = new ArrayList<>();
    }

    public void agregarProducto(Producto p, int cantidad) {
        if (p.getStock() >= cantidad) {

```

```

        p.disminuirStock(cantidad);
        for (int i = 0; i < cantidad; i++) {
            productos.add(p);
        }
    } else {
        System.out.println("✗ Stock insuficiente para " +
p.getNombre());
    }
}

public void calcularTotal() {
    total = productos.stream().mapToDouble(Producto::getPrecio).sum();
}

public void mostrarFactura() {
    System.out.println("\n=== Factura ===");
    System.out.println("Cliente: " + cliente);
    for (Producto p : productos) {
        System.out.println("- " + p.getNombre() + " $" + p.getPrecio());
    }
    System.out.println("Total: $" + total);
    System.out.println("=====");
}
}

```

```

--- MENU TIENDA DREAMS ---
1. Registrar producto
2. Registrar cliente
3. Mostrar inventario
4. Registrar venta
0. Salir
Seleccione una opción: 3

--- Inventario ---
1. Moto Royal Enfield | $93000.00 | Stock: 1
2. Moto Royal Enfield | $86000.00 | Stock: 1

--- MENU TIENDA DREAMS ---
1. Registrar producto
2. Registrar cliente
3. Mostrar inventario

```



```
--- Inventario ---  
1. Moto Royal Enfield | $93000.00 | Stock: 1  
2. Moto Royal Enfield | $86000.00 | Stock: 1  
Seleccione producto: 2  
Cantidad: 1  
¿Agregar otro producto? (s/n): s
```

```
--- Inventario ---  
1. Moto Royal Enfield | $93000.00 | Stock: 1  
2. Moto Royal Enfield | $86000.00 | Stock: 0  
Seleccione producto: 2  
Cantidad: 3  
? Stock insuficiente para Moto Royal Enfield  
¿Agregar otro producto? (s/n):
```